

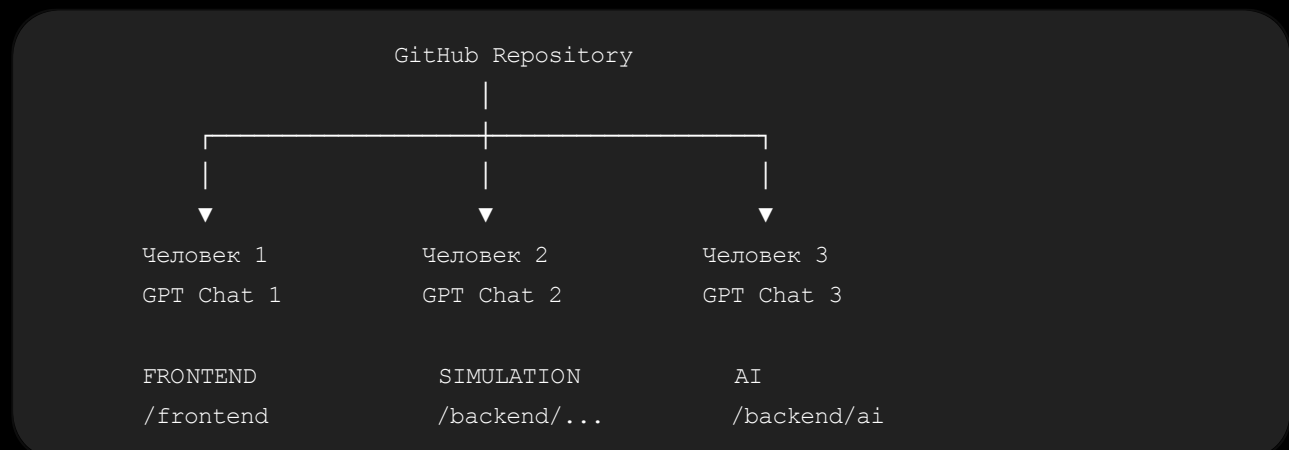
Да. **GPT Pro** сможет написать практически весь проект, если правильно разделить работу и дать каждому чату четкие границы. Он способен создать структуру проекта, компоненты, FastAPI, simulation engine, OpenAI-интеграцию, тесты, Docker/README, исправлять ошибки по логам и довести модуль до рабочего состояния.

Но есть важный момент: **три отдельных чата GPT сами между собой не общаются**. Поэтому связующим звеном должен быть GitHub. Каждый GPT работает только в своей папке и по одному заранее зафиксированному API-контракту. Вы втроем в основном будете запускать команды, проверять результат, отправлять GPT ошибки и делать pull request/merge.

Ниже я бы буквально отправил это вашей команде и вставил в три GPT-чата.

Общая схема работы

У нас есть **3 человека + 3 отдельных GPT Pro чата**.



Каждый чат должен знать:

Чат 1 не знает, что написал Чат 2.
Чат 2 не знает, что написал Чат 3.
Чат 3 не знает, что написал Чат 1.

Поэтому единственный источник правды:

1. GitHub
2. структура проекта
3. API contract
4. dataset

Перед началом

Всем троим надо иметь одну копию репозитория.

Bash

```
git clone <repo>
cd akim-ai
```

Создаем ветки:

Bash

```
git checkout -b feature/frontend
```

Bash

```
git checkout -b feature/simulation
```

Bash

```
git checkout -b feature/ai
```

Никто не кодит напрямую в `main`.

Очень важно: загрузите оба PDF во все 3 GPT-чата

Каждый GPT должен видеть:

1. PDF с описанием кейса.
2. PDF с датасетом.

Потому что именно организаторы определили правила симуляции: бюджет 100, ровно пять решений, ограничения, синергии, лаги и формулу Score. Например, итоговый Score должен учитывать средний городской показатель, самый слабый район и количество критических показателей. [Датасет районов](#)

При этом AI не должен самостоятельно выдумывать цифры — по условиям кейса LLM получает уже рассчитанные результаты и **объясняет их**, а вычисления выполняет deterministic simulation engine. [Датасет районов](#)

ОБЩИЙ PROMPT ДЛЯ ВСЕХ ТРЕХ GPT

Этот текст вставьте **первым сообщением во все три чата**.

Мы участвуем в хакатоне и создаем проект **Akim AI — AI-симулятор управления городом**.

Над одним GitHub-репозиторием одновременно работают 3 человека, каждый со своим отдельным GPT Pro чатом.

Поэтому проект намеренно разделен на независимые модули.

Ты являешься одним разработчиком внутри команды и должен строго соблюдать границы своей зоны ответственности.

Основная идея проекта

Пользователь становится виртуальным акимом Астаны.

Он получает:

- бюджет 100 условных единиц;
- данные по 5 районам;
- 14 возможных мероприятий;
- должен принять ровно 5 решений.

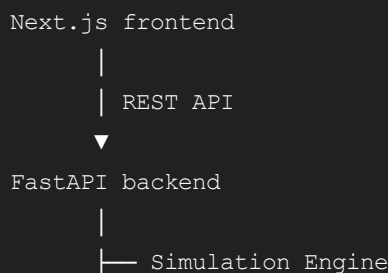
Система рассчитывает влияние решений на:

- транспорт;
- экологию;
- социальную инфраструктуру;
- безопасность;
- городские сервисы.

После этого deterministic simulation engine рассчитывает **Astana Quality of Life Score**, а LLM анализирует уже рассчитанный результат и объясняет сильные стороны, риски, последствия и возможные улучшения.

В проекте используются официальные формулы, показатели, стоимости, лаги, ограничения, несовместимости и синергии из приложенного организаторами датасета.

Архитектура



Технологии:

Frontend:

- Next.js
- TypeScript
- Tailwind CSS
- shadcn/ui
- Recharts

Backend:

- Python
- FastAPI
- Pydantic

AI:

- OpenAI API

Главное правило командной работы

Не изменяй файлы другого разработчика.

Работай только внутри своей назначенной директории.

Если тебе необходима функциональность другого модуля, не реализуй ее сам. Используй API contract или mock.

Не меняй формат API самостоятельно.

Если считаешь, что API нужно изменить — сначала объясни необходимость изменения и предложи новую схему, но не меняй ее самовольно.

Общие файлы

Следующие файлы считаются общими и не должны изменяться без необходимости:

```
/shared/api-contract.md  
/README.md  
/docker-compose.yml  
/backend/app/main.py
```

Главным интегратором проекта является разработчик Simulation/Backend.

Принцип разработки

Пиши production-like код, но без ненужного enterprise-overengineering.

Нам нужен надежный хакатонный MVP, который легко запустить.

Приоритет:

1. Работоспособность.
2. Корректность формул.
3. Простота интеграции.
4. Красивое демо.
5. Дополнительные функции.

Перед написанием кода сначала изучи приложенные PDF и существующий репозиторий.

Никогда не придумывай значения датасета, если они уже заданы организаторами.

Не удаляй существующий рабочий функционал без причины.

Делай изменения небольшими логическими этапами.

После каждого большого этапа сообщай:

- какие файлы созданы;
- какие файлы изменены;
- как это запустить;
- как протестировать;
- какой commit message использовать.

Если возникает ошибка, сначала разберись в причине, а не переписывай весь проект.

Перед завершением своей части обязательно проверь build/tests.

Я буду выполнять твои команды на своем ПК и отправлять тебе терминал и ошибки.

Наша цель — получить полностью рабочий проект, а не просто пример кода.

GPT №1 — FRONTEND

После общего промпта первый человек отправляет это.

ТВОЯ РОЛЬ: FRONTEND DEVELOPER

Ты отвечаешь исключительно за:

`/frontend/**`

Другие разработчики параллельно пишут backend и AI.

Не редактируй их код.

Твоя задача

Создать современный интерфейс AI-симулятора управления Астаной.

Интерфейс должен выглядеть как настоящий polished hackathon product, а не как простая учебная форма.

Основные страницы

`/`

Landing / Welcome.

Показываем:

AKIM AI

Вы — аким Астаны на 5 часов.

Budget: 100

Decisions: 5

Districts: 5

[Начать управление]

`/simulator`

Главный экран.

Должны быть:

- текущий бюджет;
- количество решений `0/5` ;
- текущий Astana Quality of Life Score;
- районы;

- показатели выбранного района;
- каталог мероприятий;
- выбранные решения;
- прогноз влияния решения;
- отображение конфликтов;
- отображение синергий.

/results

Итог:

- Score before;
- Score after;
- delta;
- показатели районов before/after;
- radar chart;
- contribution каждого решения;
- AI Summary;
- Strengths;
- Risks;
- Trade-offs;
- Recommendations.

Компоненты

Желательная структура:

```
frontend/
├── app/
├── components/
│   ├── BudgetBar
│   ├── DistrictCard
│   ├── DistrictPanel
│   ├── IndicatorBar
│   ├── MeasureCard
│   ├── MeasureModal
│   ├── DecisionList
│   ├── ScoreGauge
│   ├── RadarComparison
│   ├── Timeline
│   ├── SynergyAlert
│   └── AIAalysisPanel
├── lib/
│   ├── api.ts
│   └── types.ts
```

Ты можешь улучшить организацию внутри `/frontend`, если необходимо.

Design

Стиль:

- modern government-tech;
- dark/light clean dashboard;
- красивые cards;
- progress bars;
- понятная визуализация;
- subtle animations;
- responsive.

Не перегружать экран.

Пользователь должен понимать интерфейс практически без инструкции.

API

Backend может еще не существовать.

Поэтому сначала создай typed API layer и mocks.

Основные endpoints:

```
GET /api/districts
GET /api/measures
POST /api/simulate
POST /api/scenario/finalize
POST /api/ai/analyze
```

Все вызовы API должны находиться централизованно, например:

```
frontend/lib/api.ts
```

Не размазывай `fetch()` по десяткам компонентов.

Очень важно

Не реализуй формулу Score во frontend.

Не дублируй правила simulation engine.

Backend является единственным источником истины.

Frontend только отображает рассчитанный результат.

Для разработки используй mocks, пока backend не готов.

Начни с:

1. изучения PDF;
2. изучения текущей структуры геро;
3. плана frontend;
4. создания структуры;
5. реализации MVP;
6. polish.

Не пытайся за один ответ написать весь проект. Веди меня пошагово и после каждого этапа проверяй, что приложение запускается.

GPT №2 — BACKEND + SIMULATION + INTEGRATOR

Это самый важный участник, потому что он контролирует математику и интеграцию.

Ему отправляете:

ТВОЯ РОЛЬ: BACKEND / SIMULATION ENGINE / INTEGRATOR

Ты отвечаешь за deterministic core проекта.

Твоя основная зона:

```
/backend/app/simulation/**  
/backend/app/models/**  
/backend/app/api/simulation/**  
/shared/districts.json  
/shared/measures.json  
/backend/tests/**
```

Кроме того, ты главный интегратор и контролируешь:

```
/backend/app/main.py  
/shared/api-contract.md
```

КРИТИЧЕСКОЕ ТРЕБОВАНИЕ

Все математические результаты должны вычисляться кодом.

LLM никогда не рассчитывает Score.

Используй исключительно формулы и данные организаторов из приложенного PDF.

Dataset

Перенеси официальный датасет в нормализованные JSON/Python-модели.

Не меняй значения.

Районы:

Есиль
Алматы
Сарыарка
Байконур
Нура

Индикаторы:

T1 T2
E1 E2
S1 S2
B1 B2
C1 C2

Реализовать

1. Scenario validator

Проверяет:

- бюджет ≤ 100 ;
- ровно 5 решений для finalize;
- отсутствие повторов;
- district обязателен для district-measure;
- city-measure не требует district;
- максимум две меры одного направления;
- несовместимости;

- корректность ID района;
- корректность ID мероприятия.

Validator должен возвращать понятные ошибки.

2. Effect Engine

Используй официальную формулу:

```
realized effect =  
full effect × (8 - lag) / 8
```

Значения indicators всегда ограничиваются:

```
0 <= indicator <= 100
```

3. Synergies

Реализуй все синергии из dataset.

4. Incompatibilities

Реализуй все несовместимости из dataset.

5. District Score

Реализуй веса показателей строго по официальному dataset.

6. City Score

Используй population share районов.

7. Final Score

Реализуй официальную формулу:

```
Score =  
0.7 × D_avg
```

```
+ 0.3 × min(D_d)
N_crit
```

`N_crit` = количество пар район × показатель со значением `<40`.

8. Contribution analysis

Помимо конечного результата необходимо вернуть вклад каждого мероприятия.

Например:

JSON

```
{
  "measureId": "M7",
  "districtId": "nura",
  "scoreImpact": 1.42
}
```

Чтобы frontend мог объяснять, какие решения оказали наибольшее влияние.

API

Минимум:

```
GET /api/health

GET /api/districts

GET /api/measures

POST /api/simulate

POST /api/scenario/finalize
```

`simulate` может принимать менее пяти решений для live preview.

`finalize` должен требовать строго пять.

TESTS

Это обязательно.

Первый тест:

исходный Score:

52.56

Второй тест — официальный пример организаторов:

M7 Nura
M8 Nura
M10 Nura
M12 City
M5 Saryarka

Стоимость:

95

Score должен получиться около:

56.5

Не подгоняй формулу под число. Если оно не получается, найди ошибку в реализации.

API CONTRACT

Создай:

/shared/api-contract.md

Это главный контракт между всеми тремя разработчиками.

Зафиксируй структуры request/response.

После утверждения старайся не делать breaking changes.

Начни с анализа PDF и выпиши мне:

1. все правила;
2. все формулы;
3. все мероприятия;
4. синергии;
5. несовместимости;
6. API contract;
7. структуру backend.

Только после этого начинай реализацию.

В конце каждого этапа запускай tests.

GPT №3 — AI / ANALYTICS / REPORTS

Отправляете третьему:

ТВОЯ РОЛЬ: AI ENGINEER

Ты отвечаешь только за:

```
/backend/app/ai/**  
/backend/app/api/ai/**  
/backend/app/report/**
```

Не реализуй заново simulation engine.

Не рассчитывай Score через LLM.

Все числовые результаты приходят от разработчика №2.

Главный принцип

AI является **аналитическим слоем** поверх **deterministic simulation engine**.

AI получает факты:

```
before  
after  
score  
changes  
measures  
costs  
district deltas  
critical indicators  
contributions
```

И превращает их в понятный управленческий анализ.

Реализовать

POST `/api/ai/analyze`

Input:

рассчитанный simulation result.

Output:

JSON

```
{
  "summary": "...",
  "strengths": [],
  "risks": [],
  "tradeoffs": [],
  "recommendations": []
}
```

Требования к AI

Никогда:

- не изменяет рассчитанный Score;
- не придумывает показатели;
- не утверждает эффект, которого нет в input;
- не возвращает новые fake numbers.

Все выводы должны основываться на calculated result.

AI City Council

Реализуй endpoint:

POST /api/ai/advice

Он получает:

- уже выбранные меры;
- оставшийся бюджет;
- текущие показатели;
- доступные мероприятия.

Возвращает:

JSON

```
{
  "priority": "...",
  "reason": "...",
}
```

AI может рекомендовать варианты, но backend-validator остается источником истины относительно допустимости решения.

Scenario explanation

Нужно уметь отвечать на вопросы:

- Почему Score изменился?
 - Какое решение имело наибольшее влияние?
 - Какой район сейчас самый проблемный?
 - Какие trade-offs были сделаны?
 - Как можно улучшить сценарий?
-

Executive Brief

Добавь генерацию структурированного отчета:

```
City Management Report
```

```
Initial Score
```

```
Final Score
```

```
Budget
```

```
Decisions
```

```
Major improvements
```

```
Remaining risks
```

```
Equity
```

```
Recommendations
```

Возвращай structured JSON/Markdown, чтобы frontend мог его красиво показать или экспортировать.

Работа без готового Simulation Engine

Пока backend разработчика №2 не готов, используй fixtures/mocks внутри своей зоны.

Не создавай альтернативный simulation engine.

Когда настоящий backend будет готов, просто замени mock на его response format.

Сначала:

1. изучи PDF;
2. изучи API contract;
3. предложи schema AI endpoints;
4. создай prompts;
5. добавь structured outputs;
6. tests;
7. integration.

Не меняй чужие директории.

Как вам работать параллельно на практике

После запуска каждый GPT делает свое:

День/час 1

GPT 1

Frontend skeleton + UI

GPT 2

Dataset + formulas + API

GPT 3

AI prompts + structured output

Никто никого не ждет.

GPT 1

использует:

mock API

GPT 3

использует:

mock simulation response

GPT 2

делает настоящий API.

Потом первый checkpoint

GPT №2 пушит:

Bash

```
git add .  
git commit -m "feat: implement simulation engine"  
git push origin feature/simulation
```

После проверки merge в `dev`.

Затем Frontend обновляется

Участник №1:

Bash

```
git checkout feature/frontend  
git fetch origin  
git merge origin/dev
```

И говорит GPT:

Backend уже появился. Изучи новый `/shared/api-contract.md` и замени frontend mocks на настоящий API. Не меняй backend.

AI делает то же самое

Bash

```
git checkout feature/ai  
git fetch origin  
git merge origin/dev
```

И говорит GPT:

Simulation API теперь готов. Изучи его response schema и подключи AI слой. Не меняй simulation engine.

Правило, которое спасет вам проект

Если GPT №1 внезапно говорит:

«Мне нужно немного изменить backend...»

Не давайте ему этого делать.

Пишите:

Не меняй backend. Скажи, какого API тебе не хватает, и я передам это разработчику backend.

Точно так же GPT №3.

Иначе через несколько часов у вас появятся:

```
simulation_engine.py
simulationEngine.ts
calculator.py
score.ts
final_score.py
```

и пять разных реализаций одной формулы.

Кто что может трогать

Файл	№1 Frontend	№2 Simulation	№3 AI
/frontend/**	✓	✗	✗
/backend/app/simulation/**	✗	✓	✗
/backend/app/models/**	✗	✓	✗
/backend/app/ai/**	✗	✗	✓
/backend/app/report/**	✗	✗	✓
/shared/districts.json	✗	✓	✗
/shared/measures.json	✗	✓	✗
/shared/api-contract.md	read	✓ owner	read

main.py



✓ owner



Что делать вам самим, а что отдавать GPT

Ваша работа должна быть минимальной.

Вы:

- создаете GitHub;
- создаете ветку;
- запускаете команды;
- копируете ошибки GPT;
- проверяете глазами;
- делаете merge;
- проверяете итоговое демо.

GPT:

- пишет код;
- создает файлы;
- устанавливает зависимости;
- пишет тесты;
- исправляет ошибки;
- рефакторит;
- подключает API;
- пишет документацию.

По сути, при таком подходе вы действительно можете почти полностью **вайбкодить** весь проект втроем.

Самое главное — не давать каждому GPT команду типа:

«Сделай весь наш проект».

Вместо этого:

```
GPT 1 = Product/UI
GPT 2 = Mathematical core
GPT 3 = AI intelligence
```

А GitHub + `api-contract.md` являются «языком», через который эти три независимых GPT понимают друг друга.

Я бы еще сделал **четвертый отдельный GPT-чат у одного из вас — “Lead / Reviewer”**, который вообще почти не пишет код: ему показываете структуру репозитория, PR и ошибки интеграции, а он следит, чтобы три вайбкодера не начали дублировать друг друга. Это особенно полезно последние часы перед защитой.

Источники